

Visual Perception and Robot Learning Algorithms

- 1 This lecture explains how algorithms represent objects, estimate motion and generate actions. We compare model inputs, internal mechanisms, training objectives and inference procedures.
- 2 We cover YOLO, SAM 2, FoundationPose, ORB-SLAM3, OpenVLA, π_0 and Diffusion Policy, explaining the input, mechanism and output of each method.
- 3 For every method, identify its output representation, assumptions and evaluation metric. Published results are meaningful only together with the dataset and experimental protocol.

DETECTION AND SEGMENTATION

YOLO · SAM 2

OBJECT POSE AND LOCALIZATION

**FoundationPose
ORB-SLAM3**

ACTION GENERATION

**OpenVLA · π_0
Diffusion Policy**

Perception, localization and action

- 1 For each method, connect the input representation to the output and explain how training or optimization produces the estimate. Then examine the main source of error and the evidence used to evaluate it.
- 2 We begin with YOLO object detection, then study segmentation, rigid-object pose and camera localization. The second half compares two vision-language-action models and a demonstration-based manipulation policy.
- 3 Segmentation, pose estimation, localization and action generation solve different mathematical problems. We compare their mechanisms and evaluation protocols without treating their scores as one shared ranking.

Topic	Main output
YOLO	Object boxes and class scores
SAM 2	Object masks
FoundationPose	Rigid-object pose
ORB-SLAM3	Camera trajectory and map
OpenVLA	Tokenized robot actions
π_0	Continuous action chunks
Diffusion Policy	Demonstration-based action sequences
Comparison	Assumptions, metrics and failure modes

Learning, geometry and generative action models

- 1 YOLO predicts class-labelled boxes from image features. SAM 2 predicts prompted masks with temporal memory. FoundationPose refines and ranks rendered pose hypotheses. All three learn visual representations but produce different outputs.
- 2 ORB-SLAM3 relies on feature correspondences and geometric optimization. It jointly estimates camera motion and a sparse map, using multi-view constraints and optional inertial measurements to reduce uncertainty.
- 3 OpenVLA casts action prediction as token generation. A vision-language backbone processes images and instructions, and a decoder maps predicted tokens to a numerical action representation.
- 4 π_0 and Diffusion Policy generate continuous action sequences. π_0 learns a velocity field with flow matching, while Diffusion Policy learns conditional denoising. Both represent a distribution of possible future actions.

Method	Main inference mechanism
SAM 2	Prompt-conditioned masks with temporal memory
FoundationPose	Pose hypotheses, rendering, refinement and ranking
ORB-SLAM3	Feature matching and geometric optimization
OpenVLA	Autoregressive generation of action tokens
π_0	Continuous action sampling with flow matching
Diffusion Policy	Conditional denoising of future action sequences

The method families differ in output representation, learned components and inference procedure. Details follow in the corresponding sections.

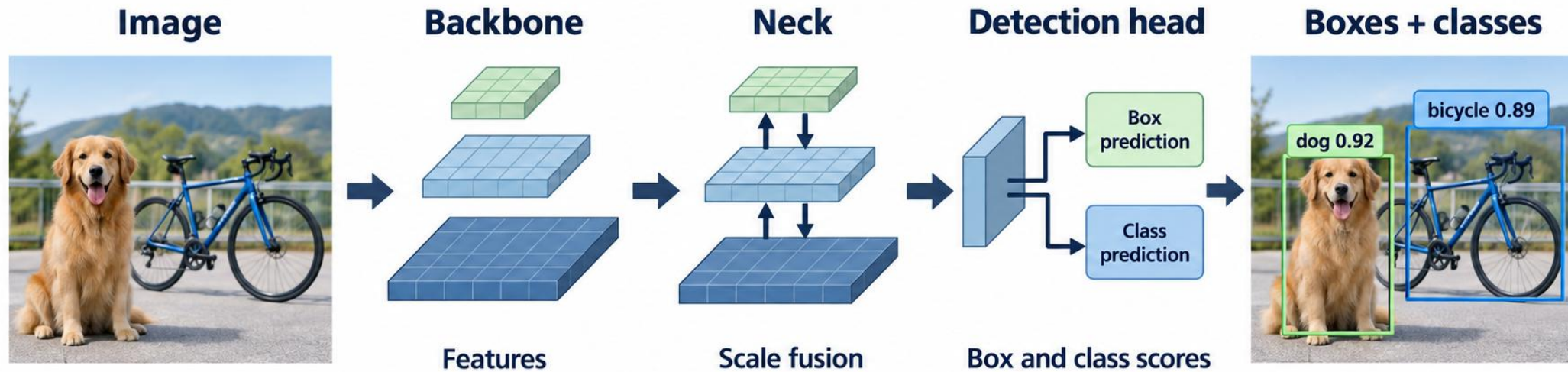
Perception, localization and action outputs

- 1 YOLO predicts object boxes and category scores. SAM 2 identifies the pixels of a prompted object. FoundationPose estimates object rotation and translation relative to a camera. These outputs describe different properties of an observation.
- 2 ORB-SLAM3 estimates a sequence of camera poses together with static 3D landmarks. Its output includes temporal and spatial relationships, so evaluation must consider trajectory consistency as well as individual frame estimates.
- 3 OpenVLA, π_0 and Diffusion Policy predict robot commands. The control interface must interpret those commands in the right action space and update them as new observations arrive.

Method	Output representation	Evaluation
SAM 2	Target mask in image coordinates	Region overlap and boundary quality
FoundationPose	Object rotation and translation	Pose error and benchmark recall
ORB-SLAM3	Camera trajectory and landmark map	ATE, drift and tracking loss
VLA / Diffusion Policy	Action or future action sequence	Task score, completion and robustness

Each output representation requires a matching error measure and an explicit evaluation protocol.

YOLO: object detection with a single network



Conceptual teaching schematic. Architecture source: Ultralytics, docs.ultralytics.com/guides/yolo-architecture

- 1 YOLO means You Only Look Once. A single network predicts bounding boxes and class scores from an image. Each detection describes an object category, its image location and a confidence score.
- 2 In YOLO I I, the backbone extracts features at different resolutions. The neck fuses these features across scales, helping the detector combine fine spatial detail with broader visual context.
- 3 The detection head predicts box geometry and class scores in separate branches. YOLO I I uses C3k2 blocks and C2PSA attention. Its detection outputs come from three scales with strides 8, 16 and 32.

YOLO II: box decoding and non-maximum suppression

1

YOLO II uses an anchor-free detection head. It predicts distances from feature-map points to the four box edges. These reference points provide locations without predefined anchor-box sizes.

2

For each edge distance, the head predicts a discrete distribution. The expected value decodes the distance into a box coordinate. Class scores indicate which trained categories the box may contain.

3

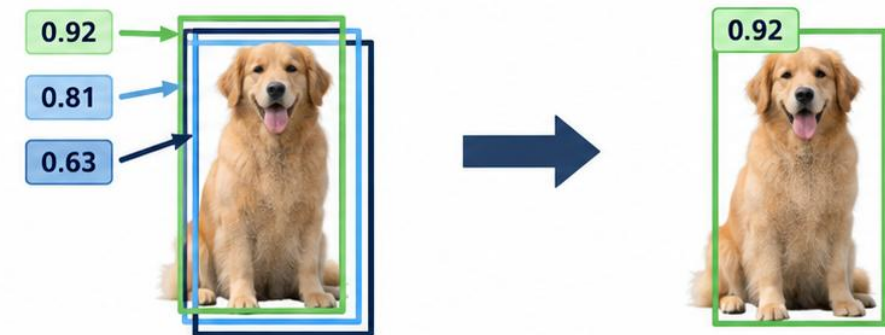
Confidence filtering first removes weak candidates. NMS then keeps the highest-scoring box and suppresses strongly overlapping boxes of the same class. It repeats this process for the remaining candidates.

4

IoU measures overlap as intersection area divided by union area. A low NMS IoU threshold removes more boxes and can suppress nearby objects. This NMS-based workflow describes YOLO II detection.

IoU = intersection area / union area

NMS (Non-Maximum Suppression)



Illustrative scores: NMS retains the 0.92 candidate and removes overlapping duplicates.

Sources: Ultralytics architecture guide and utils.nms reference.
docs.ultralytics.com/reference/utils/nms

YOLO11: training objectives and detection performance

- 1 Training assigns ground-truth boxes to candidate locations. Classification loss supervises category scores. IoU-based box loss improves geometry, while Distribution Focal Loss supervises the edge-distance distributions.

Model	COCO mAP50–95	Parameters	T4 TensorRT 10
YOLO11n	39.5	2.6 M	1.5 ± 0.0 ms
YOLO11s	47.0	9.4 M	2.5 ± 0.0 ms
YOLO11m	51.5	20.1 M	4.7 ± 0.1 ms

Ultralytics YOLO11 detection benchmark, input size 640 pixels. docs.ultralytics.com/models/yolo11

- 2 COCO mAP50–95 averages precision across categories and IoU thresholds from 0.50 to 0.95. Higher thresholds demand tighter boxes. Report precision and recall as well to explain false detections and missed objects.
- 3 Larger variants improve the published benchmark score while increasing computation. These latency figures depend on the stated GPU and inference engine. Small, occluded or unfamiliar objects remain important failure cases.

SAM 2: prompt-conditioned object segmentation

1 A prompt specifies the target region through positive points, negative points, a bounding box or a mask. The model uses this spatial guidance to predict object membership at each pixel.

2 Look at Figure 1 on the right: a box, points or a mask on one frame identifies the target, and then a mask continues through the later video frames.

3 A bounding box can contain several touching or overlapping objects. Positive points indicate the desired foreground, while negative points reject unwanted regions. Prompt quality directly influences which instance the model segments.

4 SAM 2 predicts a target mask rather than a semantic class label. Object identity comes from the prompt and temporal state. Assigning category names requires a separate recognition component or external annotation.

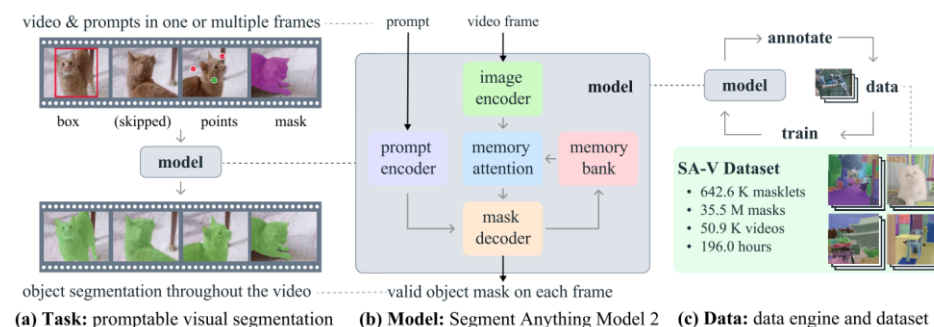
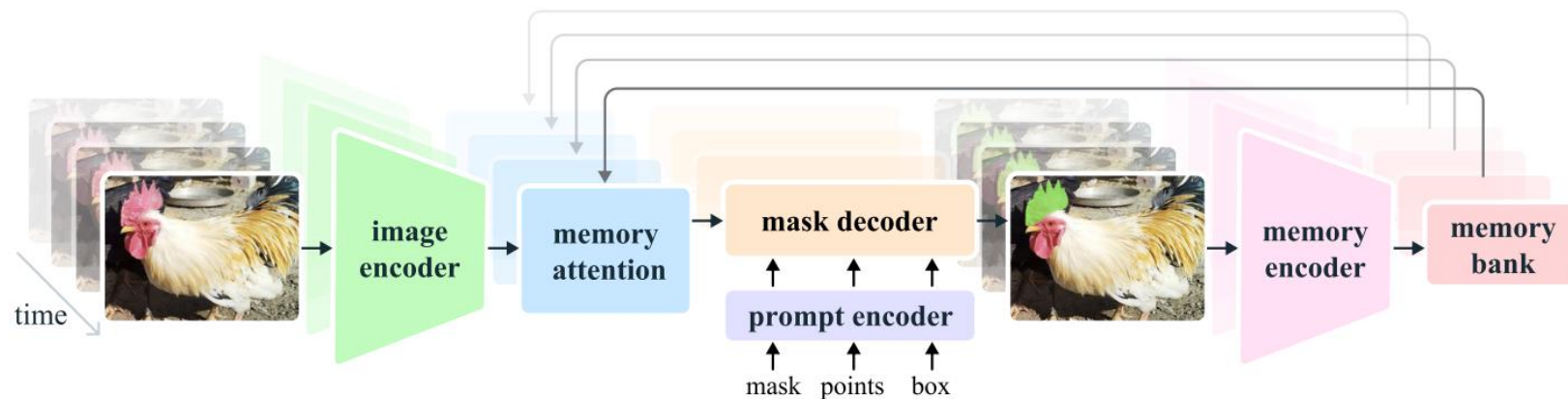


Figure 1 We introduce the Segment Anything Model 2 (SAM 2), towards solving the promptable visual segmentation task (a) with our foundation model (b), trained on our large-scale SA-V dataset collected through our data engine (c). SAM 2 is capable of interactively segmenting regions through prompts (clicks, boxes, or masks) on one or multiple video frames by utilizing a streaming memory that stores previous prompts and predictions.

Ravi et al., SAM 2, Fig. 1.

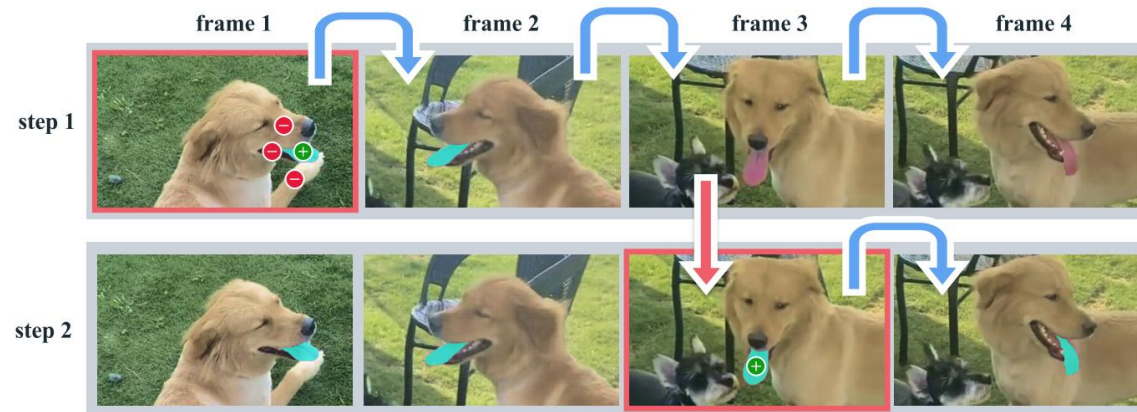
SAM 2: encode, remember, decode



Ravi et al., SAM 2, Fig. 3. Follow the figure from left to right.

- 1 Follow the figure from left to right. The image encoder sees the current frame, and memory attention then asks where the object we already selected is now, using what was stored from earlier frames.
- 2 The prompt encoder injects the click, box or mask into the network, and the mask decoder outputs the region that belongs to that object identity on this frame.
- 3 SAM 2 is not independent per-frame segmentation. Memory is what makes video tracking work, and that is also why a wrong mask can infect later frames if we never correct it on a visible frame.

SAM 2: memory helps, but errors propagate

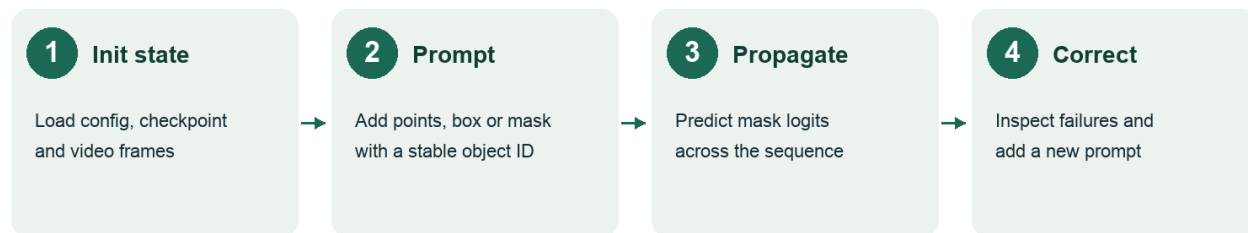


Ravi et al., SAM 2, Fig. 2.

- 1 An initial prompt selects the target on one frame. Memory-conditioned inference then propagates that identity through the video, using stored information together with the current image features.
- 2 A corrective prompt on a later frame supplies new spatial evidence. The model updates the target representation and can propagate the correction to other frames, reducing the need for independent manual annotation.
- 3 During complete occlusion, the current image contains little direct evidence of the target. Memory can preserve identity, but inaccurate predictions may propagate through time. Corrections are most informative when part of the target becomes visible again.

SAM 2: inference is a loop

- 1 Video inference maintains a state containing frame features, prompts and object identities. Initialization prepares the sequence, prompting specifies a target, and propagation repeatedly predicts masks while updating temporal information.
- 2 Point labels 1 and 0 indicate foreground and background guidance. Each predicted mask must remain associated with its frame index and object ID. This association makes temporal consistency and identity errors measurable.



SAM 2 annotation and correction example. Ravi et al., Fig. 11 (detail).

Core API: `init_state`, then `add_new_points_or_box`, then `propagate_in_video`.

SAM 2: published video-segmentation results

First-frame prompts on 17 zero-shot video datasets. J&F combines overlap and boundary quality.

Method	1 click	5 clicks	GT mask
SAM + XMem++	56.9	70.6	72.7
SAM + Cutie	56.7	72.2	74.1
SAM 2	64.7	77.6	79.3

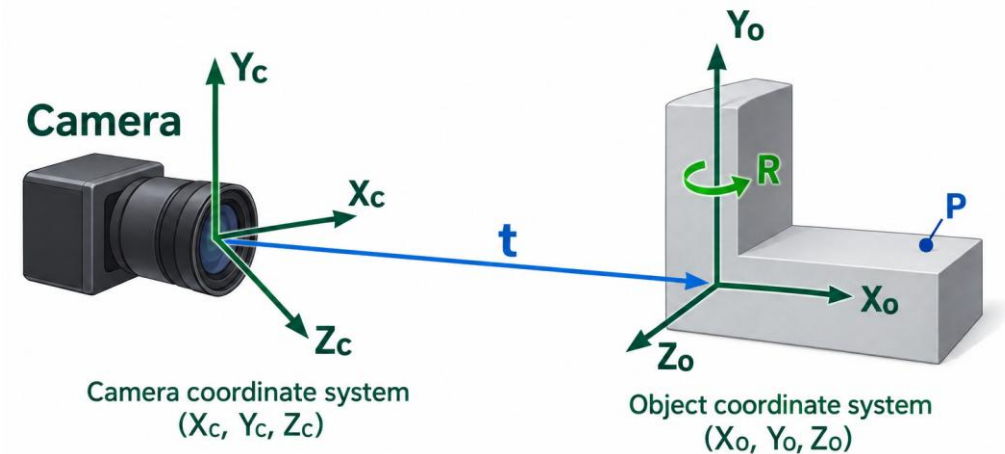
Ravi et al., SAM 2, Table 4.

1 With one click, SAM 2 is 64.7, against about 57 for the SAM-plus-tracker baselines. Stronger prompts help every method, but SAM 2 stays ahead in this table.

2 The benchmark evaluates zero-shot transfer across seventeen video datasets. J measures region overlap and F measures boundary quality. Results should be compared under the same prompt type, annotation budget and dataset aggregation rule.

FoundationPose: six degrees of freedom

- 1 Six degrees of freedom means three translations and three rotations. The rotation is stored as a 3×3 matrix R , but it still only represents three independent angles.
- 2 An image location provides two coordinates but does not uniquely determine an object's 3D orientation or depth. A rigid pose specifies an object coordinate frame relative to the camera, including both its origin and axis directions.
- 3 The rotation matrix has nine entries constrained by orthogonality and determinant $+1$, leaving three rotational degrees of freedom. Translation adds three independent parameters. The combined transform preserves distances between object points.
- 4 X in the camera frame equals R times X in the object frame plus t . Every reported pose needs a named frame, and it needs explicit units, millimetres or metres.



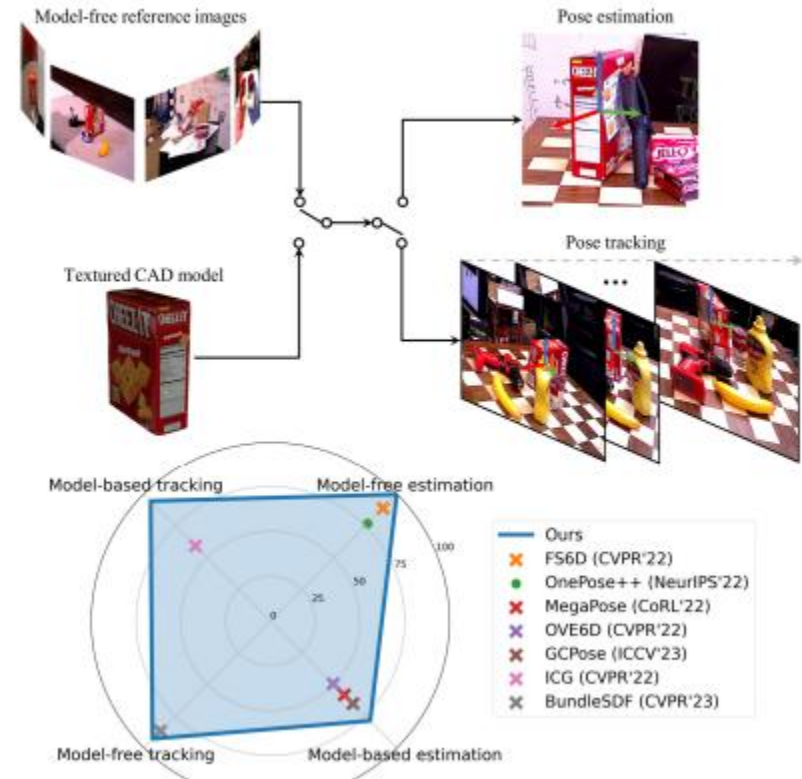
$$X_c = R X_o + t$$

3 translation parameters + 3 rotation parameters

Teaching schematic of an object-to-camera rigid transform. R changes orientation and t locates the object origin in camera coordinates.

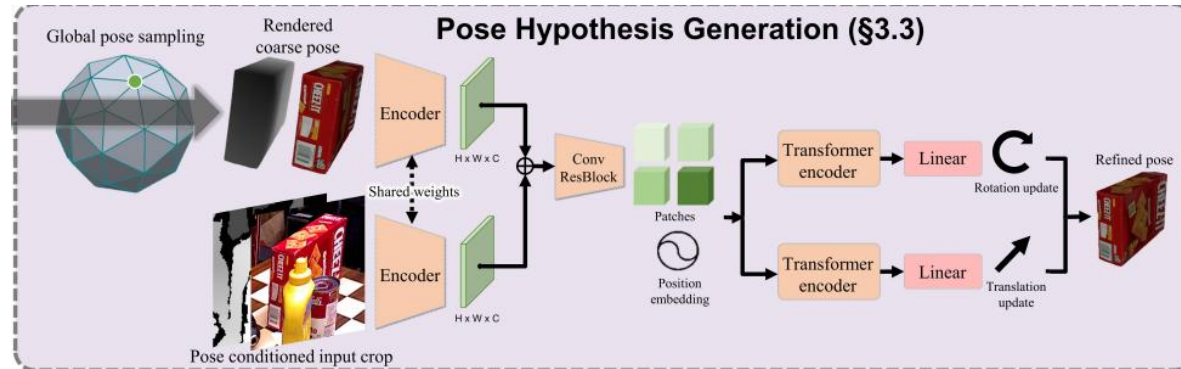
FoundationPose: novel objects still need object information

- 1 Figure 1 has two input routes into the same estimator: a CAD mesh, or a set of reference views that build a neural object model, then pose estimation and tracking.
- 2 Model-free does not mean no object information. It only means there is no CAD file; the method still builds a neural object model from those reference views before it can score a pose.
- 3 Shared inputs for both routes are RGB-D, camera intrinsics, and an object mask, which SAM 2 can help provide when the object is new on the table.
- 4 If millimetres and metres are mixed, or if RGB and depth are misaligned, the overlay on the image can look plausible and the pose can still be wrong by a large amount.



Wen et al., FoundationPose, Fig. 1. Model-free is not “no object model”.

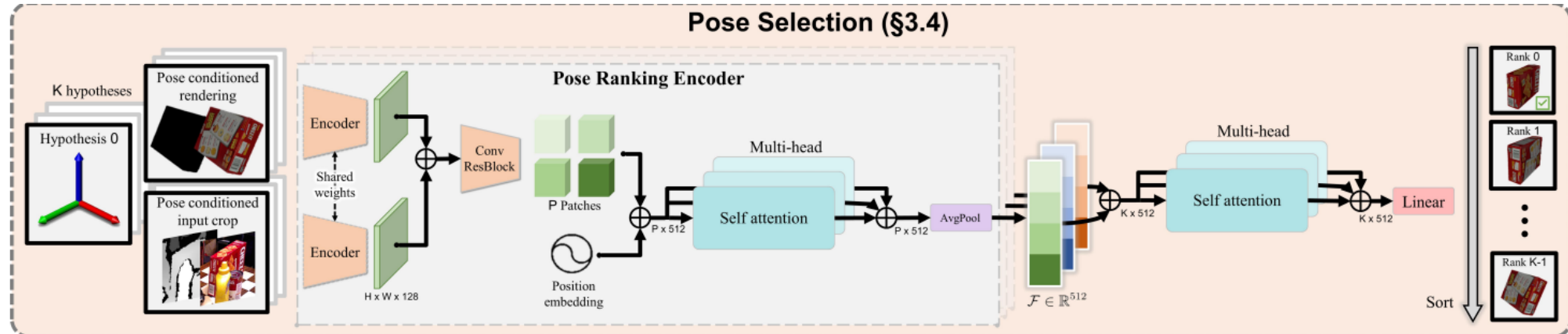
FoundationPose: generate hypotheses, then refine



Wen et al., FoundationPose, Fig. 2. This is the core mechanism.

- 1 We do not regress one pose in a single shot. Sample several hypotheses first, because a new object, occlusion, or symmetry can look consistent with more than one starting orientation.
- 2 For each hypothesis, render the object under that pose and compare the rendered RGB-D crop with what the camera actually observed in that window of the image.
- 3 Think of moving a virtual part until it matches the camera view. Refinement is only a local correction, so a bad start can still converge to the wrong pose and look smooth.
- 4 This loop is the mechanism : sample, render, compare, then apply a small rotation and translation update, and repeat until ranking picks one.

FoundationPose: rank the refined candidates



Wen et al., FoundationPose, Fig. 2. The language model helps training data, not test-time coordinates.

- 1 After refinement we still have several candidates rather than one answer. Ranking scores how well each rendered pose matches the current RGB-D observation and then picks the best.
- 2 Improving one candidate and choosing among several candidates are two different jobs. A method can refine well and still pick the wrong hypothesis if ranking is weak.
- 3 The language model does not output the test-time pose; it only helps generate varied training appearances. Symmetric parts may have two rotations that are both acceptable under the evaluation protocol.

FoundationPose: registration, tracking and initialization

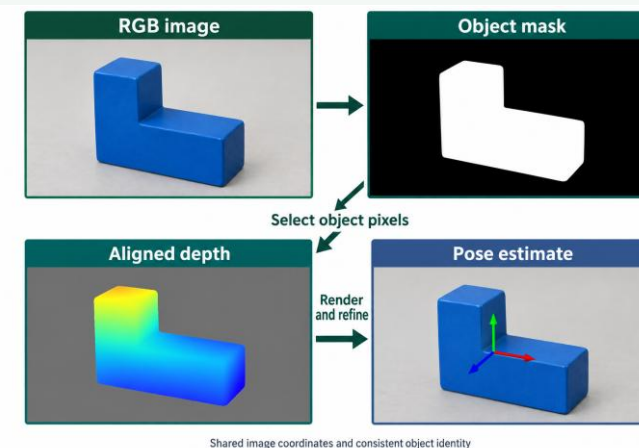
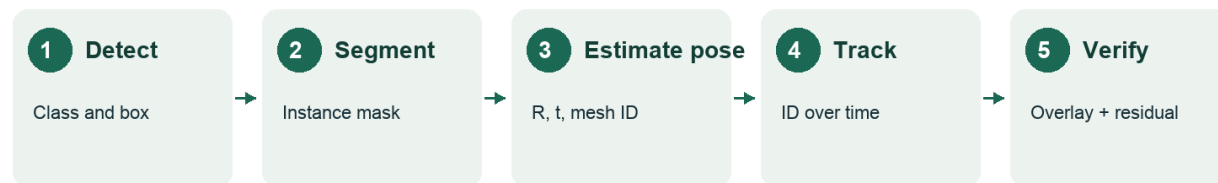
- 1 Registration estimates an initial pose by generating and evaluating multiple hypotheses. Tracking uses the previous pose as its starting point, reducing the search when consecutive frames contain only a small motion.
- 2 Local refinement can correct an approximate pose but may fail after large motion or prolonged occlusion. Reinitialization expands the search again. A temporally smooth estimate can still be geometrically incorrect.
- 3 Object geometry, depth and translation must use consistent units. Camera intrinsics and RGB–depth alignment determine how 3D points project into the image. Errors in these inputs can produce systematic residuals even when the network behaves consistently.

Aspect	Registration	Tracking
Starting information	Object information and current RGB-D	Object information, RGB-D and previous pose
Pose initialization	Multiple candidate orientations	Previous frame estimate
Computation	Refinement and ranking of candidates	Local refinement near the previous pose
Main sensitivity	Ambiguity and candidate coverage	Motion size and accumulated tracking error

Wen et al., FoundationPose. Registration and tracking share refinement but use different initial pose information.

SAM 2 and FoundationPose: mask, depth and identity

- 1 A foreground mask selects the object pixels used by pose estimation. RGB and depth must refer to the same view, and the object model must describe that same target. Segmentation supplies spatial support rather than the final 3D pose.
- 2 If a mask includes background pixels, the selected depth region contains inconsistent geometry. If object identity changes between frames, the previous pose may initialize the wrong object. These input errors can resemble failures of pose refinement.
- 3 Reprojection overlays reveal whether rendered geometry agrees with the image. Translation and rotation errors require ground-truth poses and a defined coordinate convention. Symmetric objects also require an evaluation rule that treats equivalent poses consistently.



A consistent input record links image, depth, object mask, timestamp, object ID and pose convention.

Teaching schematic: matched image coordinates link a mask, depth pixels and a rigid pose.

FoundationPose: model-based pose performance

Average recall, or AR, on BOP RGB-D datasets. Higher AR is stronger benchmark recall, not millimetre error.

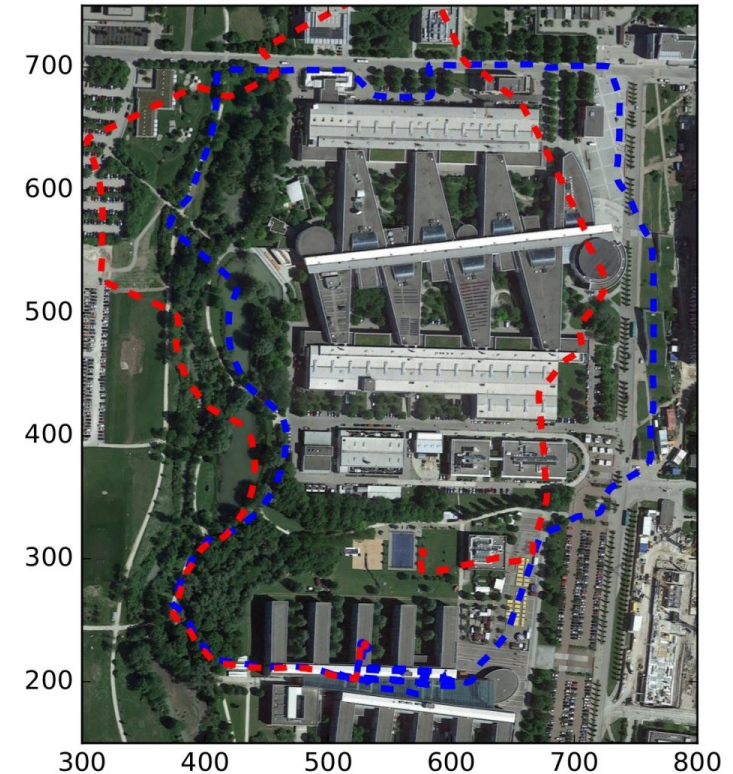
Method	LM-O	T-LESS	YCB-V
SurfEmb + ICP	75.8	82.8	80.6
MegaPose-RGBD	58.3	54.3	63.3
FoundationPose	78.8	83.0	88.0

Wen et al., FoundationPose, Table 3.

- 1 FoundationPose reports mean AR of 83.3 across these three BOP RGB-D datasets, compared with 58.6 for MegaPose-RGBD. The improvement describes recall under the paper's shared benchmark protocol.
- 2 Average recall aggregates success over pose-error thresholds under the benchmark definition. It is not a translation error in physical units. Interpret AR together with the error functions, object symmetries and dataset conditions.

ORB-SLAM3: camera localization and a reusable map

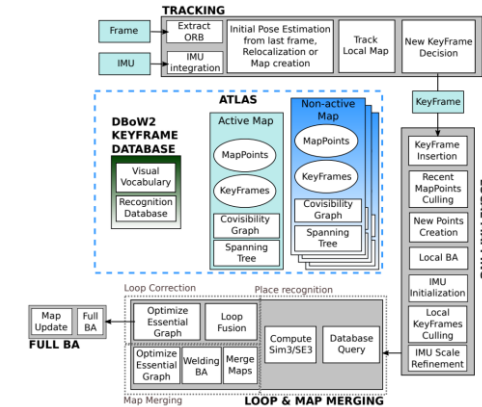
- 1 SLAM means simultaneous localization and mapping. As a camera moves, the system estimates its trajectory and the positions of visible landmarks. Camera poses and map points constrain each other through their image correspondences.
- 2 FoundationPose estimates an object relative to a camera. ORB-SLAM3 estimates the camera relative to a map. Their transforms describe different coordinate relationships, which can be composed when timestamps, units and scale are compatible.
- 3 ORB-SLAM3 is a feature-based geometric system. It supports monocular, stereo and RGB-D cameras, and visual-inertial configurations. It combines tracking with map optimization and recognition of previously visited places.
- 4 The outdoor example shows why map reuse matters. A trajectory can drift during a long trip. Revisiting mapped structure supplies extra constraints. The displayed route is an estimated camera trajectory, rather than a collision-free navigation path.



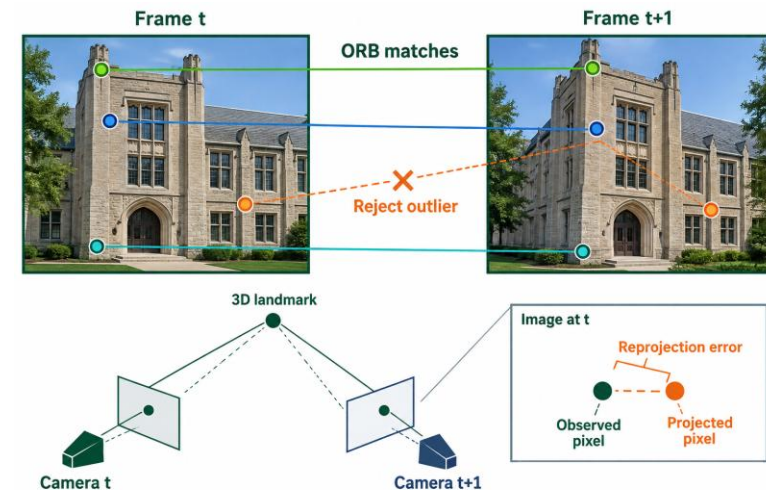
Campos et al., ORB-SLAM3, Fig. 6. Red: one session. Blue: processing multiple sessions.

ORB-SLAM3: feature matching and camera tracking

- 1 An ORB feature combines a repeatable image keypoint with a compact binary descriptor. Matching descriptors proposes that two image locations show the same physical point. Geometric checks reject matches that disagree with the estimated camera motion.
- 2 Tracking projects known 3D map points into the current image. The estimated camera pose should place each projected point close to its observed feature. Minimizing these reprojection errors turns image correspondences into a pose estimate.
- 3 A new keyframe preserves a useful viewpoint when the camera has moved enough or the current map needs more observations. Keeping selected keyframes limits computation while retaining observations that help recover scene geometry.
- 4 Repeated textures can produce plausible but incorrect matches. Motion blur removes distinctive features, and moving objects violate the static-landmark assumption. Tracking quality depends on feature visibility, geometric consistency and the distribution of matched points.

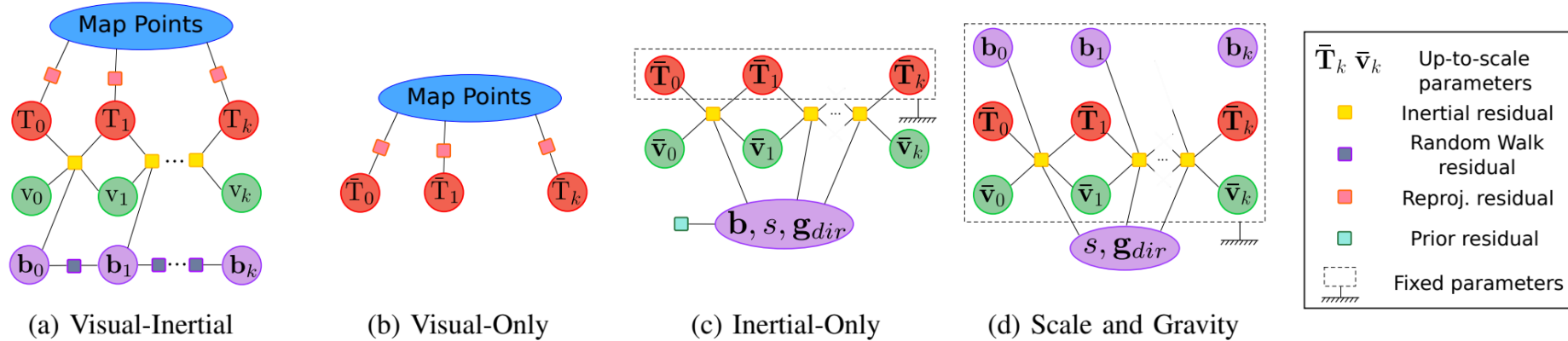


Campos et al., ORB-SLAM3, Fig. 1. System overview.



Teaching schematic: feature correspondences and reprojection error.

ORB-SLAM3: bundle adjustment and inertial constraints

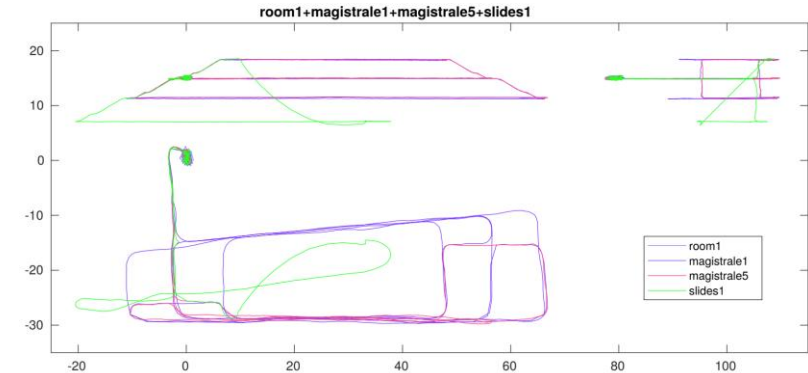


Campos et al., ORB-SLAM3, Fig. 2. Nodes represent estimated variables, and links represent residual constraints.

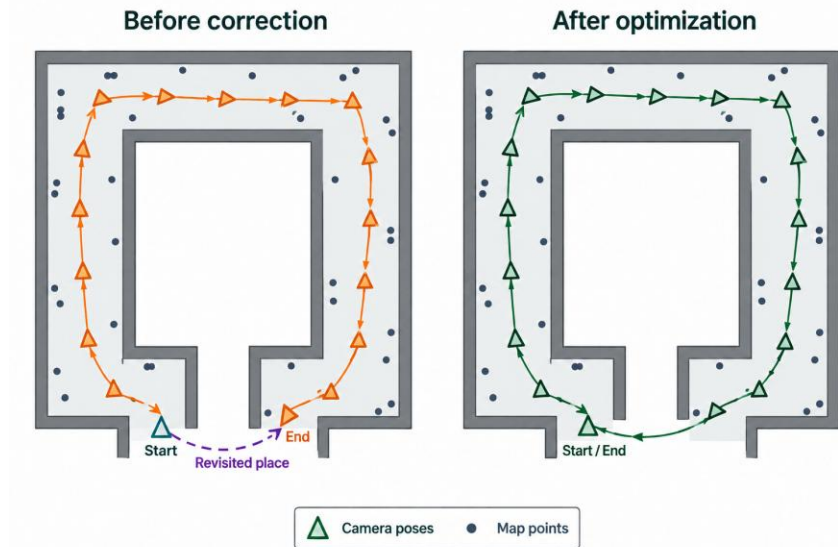
- 1 Bundle adjustment jointly refines camera poses and 3D landmarks to reduce reprojection error across keyframes. Multiple views constrain each landmark, preventing independent fits to each observation.
- 2 IMU preintegration converts angular velocity and acceleration measurements into motion constraints. Visual-inertial optimization jointly estimates velocity and sensor biases to limit drift.
- 3 Monocular geometry has unknown scale. Stereo or depth provides metric information. An IMU can recover scale with sufficiently exciting motion, but initialization and camera-IMU calibration remain essential.
- 4 Visual factors constrain landmark locations, and inertial factors constrain motion. Robust losses reduce mismatch influence. Local optimization refines a limited set of keyframes to control computation.

ORB-SLAM3: loop closure and map merging

- 1 A loop closure occurs when the robot recognizes a place that already belongs to its active map. Image retrieval proposes a candidate, and geometric verification checks that the two views really observe compatible 3D structure.
- 2 The loop supplies a constraint between distant parts of the trajectory. Graph optimization and bundle adjustment distribute the correction across camera poses and landmarks, reducing accumulated drift rather than changing only the latest frame.
- 3 The Atlas maintains multiple maps. After prolonged tracking loss, the system can start a new map. If later observations overlap an older map, it estimates their relative alignment, merges the maps and refines the shared geometry.
- 4 Perceptual aliasing occurs when different places look similar. A false loop can deform an otherwise useful map. Geometric verification must therefore support image retrieval. Evaluate relocalization and loop correctness alongside average trajectory error.



Campos et al., Fig. 5. Several sessions share a map after merging.



Teaching schematic: a verified loop corrects accumulated pose drift.

Rigid transforms: composition and frame conventions

$$\mathbf{T}_{\mathbf{WO}} = \mathbf{T}_{\mathbf{WC}} \times \mathbf{T}_{\mathbf{CO}} \quad \text{and} \quad \mathbf{T}_{\mathbf{BO}} = \text{inverse}(\mathbf{T}_{\mathbf{WB}}) \times \mathbf{T}_{\mathbf{WO}}$$

Quantity	Meaning	Typical source
$\mathbf{T}_{\mathbf{WC}}$	Camera pose in the world/map frame	SLAM, with the output direction checked
$\mathbf{T}_{\mathbf{CO}}$	Object pose in the camera frame	FoundationPose
$\mathbf{T}_{\mathbf{WB}}$	Reference frame B expressed in world/map	Frame calibration or map alignment
$\mathbf{T}_{\mathbf{BO}}$	Object pose expressed in reference frame B	Composition of the transforms above

Read $\mathbf{T}_{\mathbf{AB}}$ as the transform that maps coordinates from frame B into frame A. Multiplication follows the coordinate chain.

- 1 This convention makes it possible to combine the camera trajectory with an object pose without mixing transform directions.

- 2 Transform composition assumes common units and compatible timestamps. A rigid transform preserves distances and angles, while a similarity transform also contains scale. A monocular map needs scale alignment before composition with a metric object pose.

Notation: W = world/map, C = camera, O = object, B = another reference frame. $\mathbf{T}_{\mathbf{AB}}$ maps coordinates from B into A.

ORB-SLAM3: trajectory accuracy and evaluation limits

- 1 Absolute trajectory error, or ATE, compares estimated and reference trajectories after alignment. The table reports mean RMS ATE across six TUM-VI room sequences. Lower values indicate closer trajectories under that specific evaluation protocol.
- 2 Monocular alignment includes a scale correction, while metric configurations use rigid alignment. The rows therefore require different context. ATE summarizes trajectory disagreement after alignment and does not directly measure landmark accuracy or orientation error.

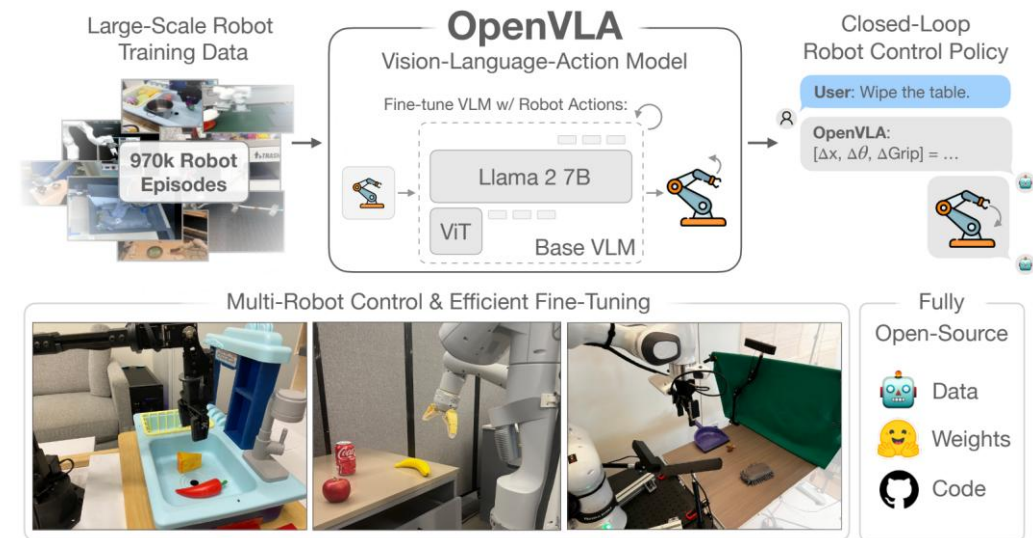
ORB-SLAM3 configuration	Mean RMS ATE	Alignment
Monocular	0.039 m	7 DoF, including scale
Stereo	0.068 m	6 DoF, metric scale
Monocular + IMU	0.011 m	6 DoF, metric scale
Stereo + IMU	0.009 m	6 DoF, metric scale

On EuRoC, the paper reports 0.035 m average RMS ATE for stereo-inertial ORB-SLAM3. Sequence length, motion, texture and sensor configuration affect the result, so errors from different datasets require separate interpretation.

Campos et al., Table IV. Median of 3 executions per sequence. Tracking loss, drift and runtime provide complementary information.

OpenVLA: images and instructions become robot actions

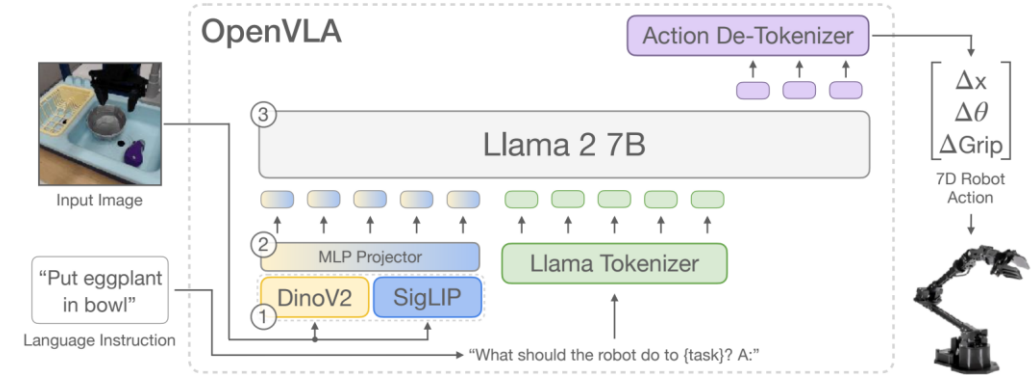
- 1 A vision-language-action model conditions on visual observations and a language instruction to predict robot actions. The language input specifies the intended behavior, while the image provides information about the current state.
- 2 OpenVLA starts from a pretrained vision-language model and trains it on 970,000 robot episodes from a curated Open X-Embodiment mixture. The original model uses a 7B language backbone and predicts a seven-dimensional end-effector action.
- 3 Those dimensions describe translation, rotation and gripper control. An action describes how the robot should move at the current step. The interface still needs the correct units, normalization and controller convention for the target robot.
- 4 Web pretraining supplies visual and language representations. Robot demonstrations align those representations with numerical actions. Transfer depends on the training distribution, action encoding and observation format, so a new embodiment may require adaptation.



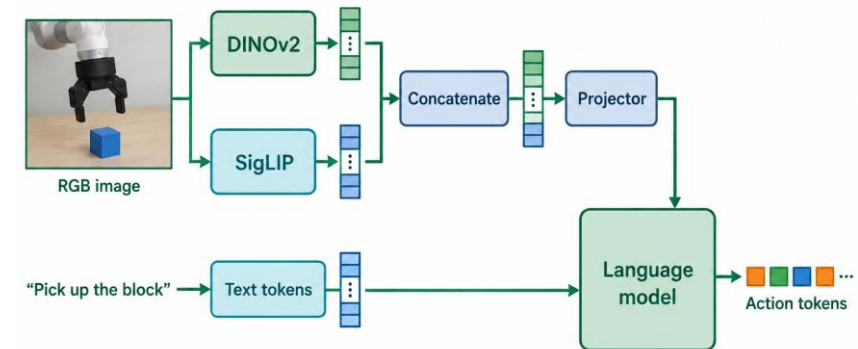
Kim et al., OpenVLA, Fig. 1. Lecture topic adapted from CIVIL 8025, slides 56–59.

OpenVLA: visual encoders and action tokens

- 1 The image passes through DINOv2 and SigLIP visual encoders. OpenVLA concatenates their patch features, combining useful spatial information with representations learned from image–language data.
- 2 An MLP projector maps the visual features into the language model embedding space. The instruction becomes language tokens. The Llama 2 backbone processes both sources of information to predict the action-token sequence.
- 3 Training uses demonstrated actions as the target sequence and applies cross-entropy loss to the action tokens. At inference, the model generates the tokens autoregressively and decodes them into continuous controller values.
- 4 This architecture reuses a language-model training interface for control. Sequential token generation adds latency, and discretization limits action resolution. These limitations motivate the continuous action-chunk approach introduced next in π_0 .



Kim et al., OpenVLA, Fig. 2. Published model architecture.

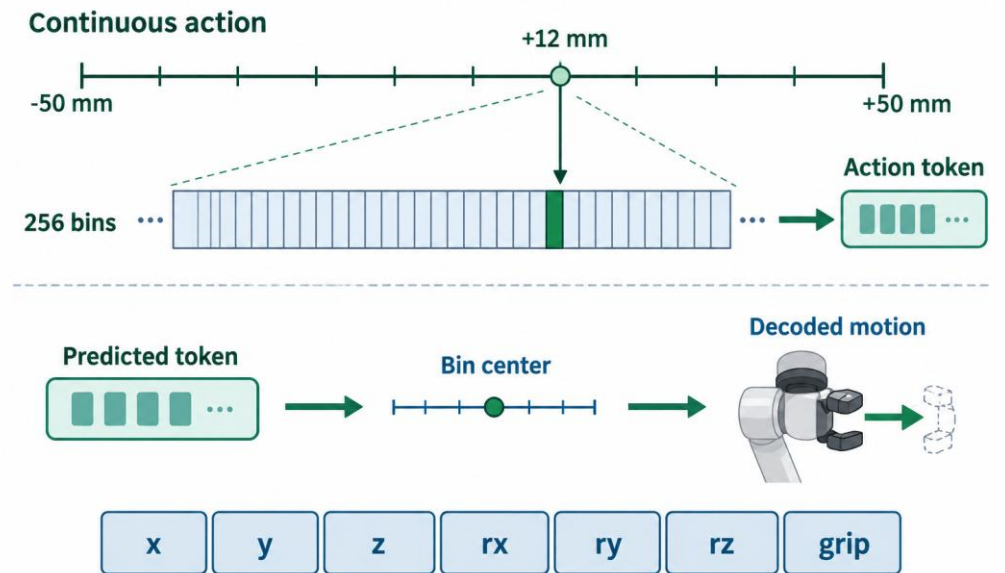


Teaching schematic: parallel visual features join language tokens.

OpenVLA: discretization and adaptation to a new robot

256 bins per action dimension bin width $\approx$ action interval / 256

- 1 OpenVLA maps each action dimension to 256 bins between the first and ninety-ninth percentiles of training actions. Percentile bounds reduce the influence of outliers on quantization.
- 2 For an illustrative 0.10 m interval, 256 equal bins give a width of about 0.39 mm. This is quantization resolution. Camera, controller and contact errors determine the final physical accuracy.
- 3 Adapting to a new robot needs compatible images, instructions and action fields. The study uses 10–150 demonstrations per task. Keep normalization consistent: the same token can decode to different values.
- 4 LoRA trains low-rank parameter matrices. In the selected test, rank-32 LoRA trains 1.4% of parameters and reaches 68.2% success, versus 69.7% for full fine-tuning. The error bars overlap.



Illustrative range; bin width is not robot accuracy.

Teaching schematic. Bounds and displacement are illustrative.

Kim et al., Sections 3.2 and 5.2–5.3, Table 1. The numerical bin-width example is illustrative, not a measured robot accuracy.

OpenVLA: published generalization results

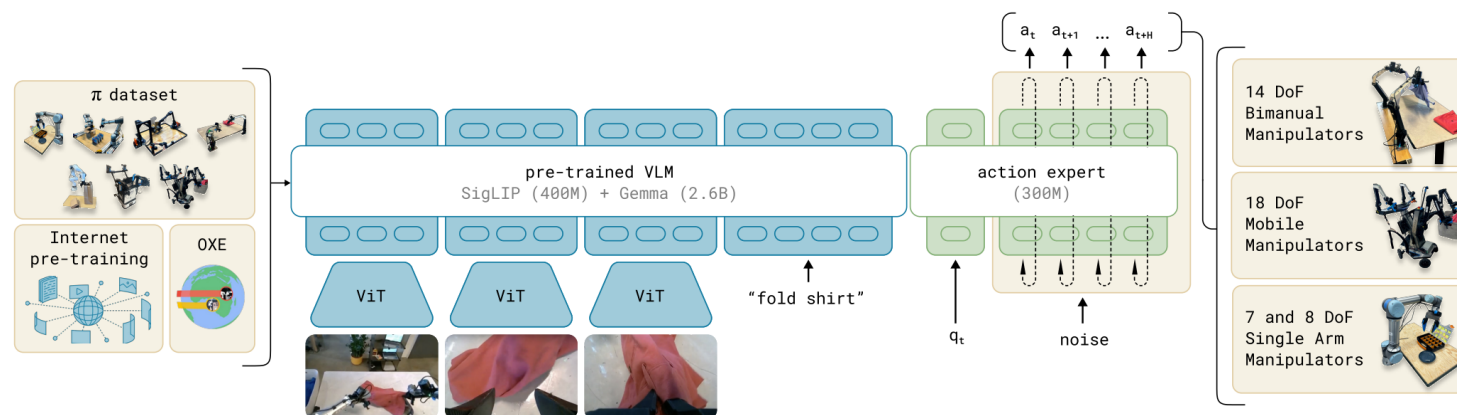
- 1 The BridgeData V2 evaluation uses 17 tasks and 10 trials per task on the WidowX robot. Tests change appearance, object position, physical properties and language instructions. Some tasks allow partial credit, so the aggregate is a task score expressed as a percentage.
- 2 OpenVLA obtains the highest aggregate result in this evaluation. Its advantage does not imply superiority on every task. Difficult semantic concepts and narrow precision tasks can favor different methods, so deployment tests should match the intended behavior.

Method	Mean success / task score	Evaluation
RT-1-X	$18.5 \pm 2.7\%$	170 rollouts
Octo	$20.0 \pm 2.6\%$	170 rollouts
RT-2-X	$50.6 \pm 3.5\%$	170 rollouts
OpenVLA	$70.6 \pm 3.2\%$	170 rollouts

Generalization evaluation varies appearance, initial state or instruction while holding other factors controlled. Matching trial distributions and reporting uncertainty make policy comparisons interpretable. Aggregate scores should be accompanied by per-task failure analysis.

Kim et al., Table 4. Mean $\pm$ standard error. The 20.0 percentage-point gap here is specific to BridgeData V2.

π_0 : a vision-language backbone with an action expert



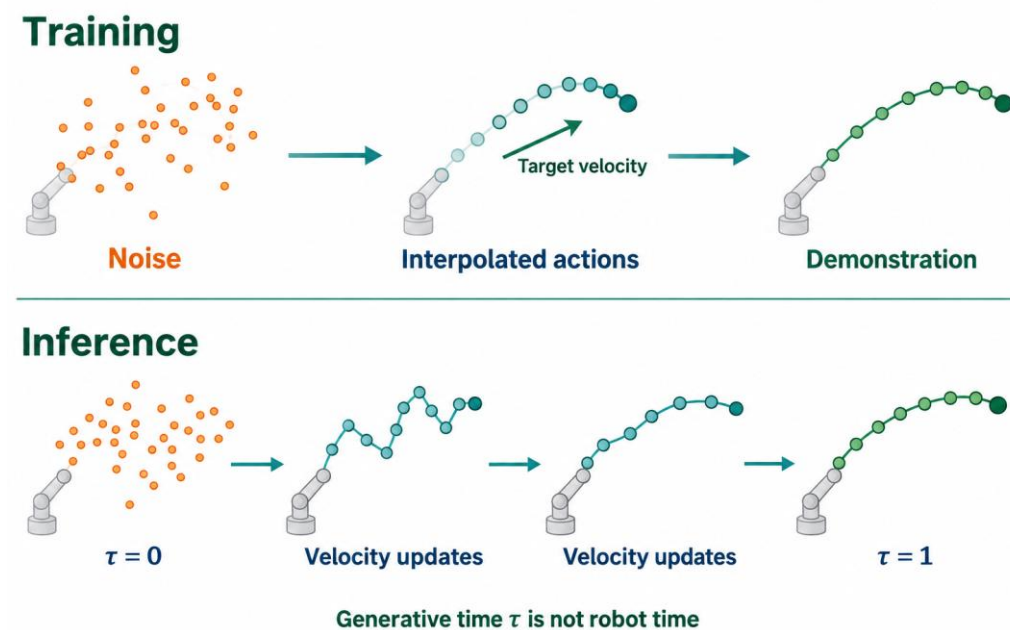
Black et al., π_0 , Fig. 3. Lecture topic adapted from CIVIL 8025, slides 60–65.

- 1 π_0 uses camera images, a language instruction and the robot's proprioceptive state, including joint angles. Robot state provides position information that an image alone may not reveal precisely.
- 2 A pretrained PaliGemma VLM encodes visual and language context. A separate 300M-parameter action expert attends to this context while processing robot state and noisy future actions.
- 3 The action expert generates continuous action chunks with conditional flow matching. Interactions among positions in the chunk let the model represent coordinated motion over time.
- 4 The paper uses 50-action chunks and demonstrates control up to 50 Hz in its setups. Chunk length, command rate and inference time are distinct quantities that together determine feedback timing.

π_0 : the flow-matching training objective

$$\mathbf{A}(\tau) = \tau \mathbf{A} + (1 - \tau) \boldsymbol{\varepsilon} \quad \text{target velocity} = \mathbf{A} - \boldsymbol{\varepsilon}$$

- 1 Let $\mathbf{A}$ be a demonstrated action chunk and $\boldsymbol{\varepsilon}$ be Gaussian noise of the same shape. Sample flow time τ between 0 and 1. $\mathbf{A}(\tau)$ is mostly noise near 0 and approaches the demonstrated actions near 1.
- 2 The network receives $\mathbf{A}(\tau)$, τ and the current observations. Its target is the velocity $\mathbf{A} - \boldsymbol{\varepsilon}$ along the interpolation path. A squared-error loss teaches the model to move noisy samples toward demonstrations.
- 3 At inference, start with noise and repeatedly update the chunk using the predicted velocity. The paper uses 10 Euler steps. Cached observation features reduce repeated computation.
- 4 Flow time τ indexes action generation, while robot time t indexes physical execution. Ten flow updates produce one action chunk. They do not mean the robot has executed ten actions.



Teaching schematic. Flow time describes generation of the action chunk.

Black et al., Section IV. Simplified notation follows the paper's linear interpolation and conditional flow-matching formulation.

π_0 : pretraining, specialization and closed-loop execution

1 Pretraining exposes the model to varied robot embodiments and manipulation situations. Post-training then uses curated demonstrations to specialize behavior for demanding tasks. Breadth helps with unfamiliar states, while consistent demonstrations improve the intended execution strategy.

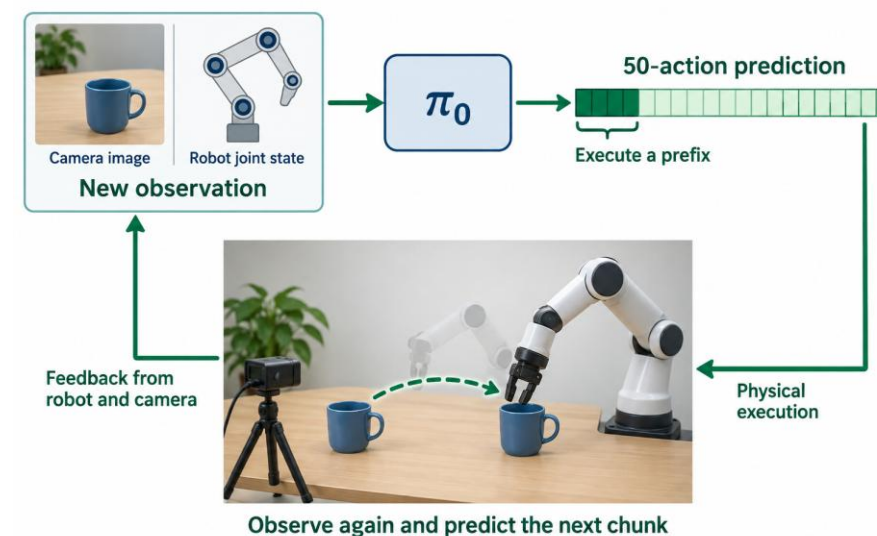
2 During execution, the system samples future commands from the current images and state. It executes a chosen portion, receives updated observations and predicts again. This closes the loop when an object shifts or a grasp does not proceed as expected.

3 A high-level planner can give intermediate instructions for the next object or subtask, while π_0 predicts detailed commands. Evaluate performance with such subgoals separately from performance under one broad instruction.

4 Specialization changes the conditional action distribution toward the target demonstration data. Coverage of initial states and recovery states affects robustness. Evaluation should separate generalization across tasks, visual conditions and robot embodiments.



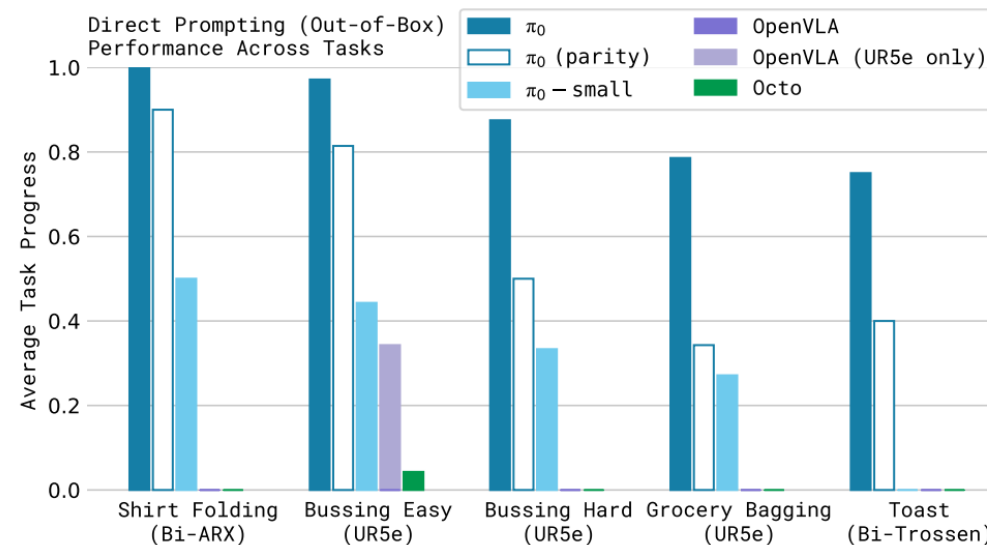
Black et al., Fig. 8. Table clearing, table setting and grocery packing.



Teaching schematic: execute a prefix, observe again, then predict.

π_0 : task progress, generalization and limitations

- 1 This evaluation measures progress on tasks such as clearing a table, folding a shirt and bagging groceries. A score of one represents full completion, while partial progress can earn an intermediate score. Read the vertical axis as task progress.
- 2 The full π_0 model outperforms the tested baselines in these tasks. The “parity” variant uses the same number of training updates as the baselines, which helps examine whether longer training alone explains the result.
- 3 The smaller model without VLM initialization also performs worse. This comparison changes both model size and pretraining, so it does not isolate a pure pretraining effect. The benchmark supports the complete design under the reported experimental setup.
- 4 Task progress is one summary of policy behavior. Completion time, recovery and intervention frequency provide additional information. A high average progress score can coexist with failures at the final state or termination step.



Black et al., π_0 , Fig. 7. Average task progress on five evaluation tasks, with partial completion credit.

Diffusion Policy: real-world Push-T performance

- 1 In Push-T, the robot pushes a T-shaped block onto a target region, then moves its end effector to a finishing zone. Success depends on final overlap, so a policy must make fine corrections and terminate at the right moment.
- 2 The table selects published variants from the same real-world experiment. End-to-end visual Diffusion Policy reaches 95% success and 0.80 mean IoU. These numbers characterize Push-T under the stated observation, control and evaluation protocol.

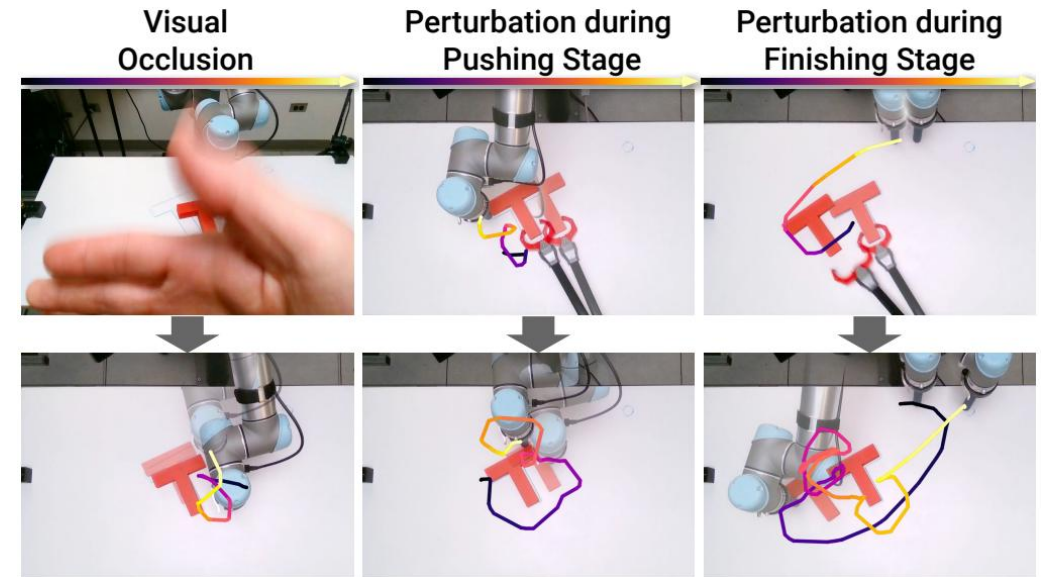
Policy variant	Success	Mean final IoU	Mean duration
IBC, position control	0%	0.14	56.3 s
LSTM-GMM, position control	20%	0.24	47.3 s
Diffusion Policy, CNN E2E	95%	0.80	22.9 s
Human demonstrations	100%	0.84	20.3 s

The paper evaluates learned variants over 20 trials in this real-world setting. The human row describes demonstrations, so it is a reference rather than an identically sampled policy trial. Report trial count and success definition with every comparison.

Chi et al., Table V. Success uses the minimum final IoU of the demonstration set as its threshold. Duration is in seconds.

Diffusion Policy: robustness and distribution shift

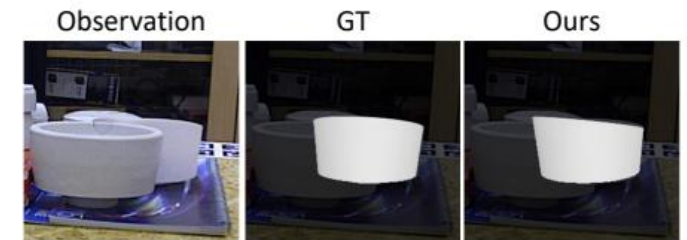
- 1 The conditional policy learns the relationship between observation histories and demonstrated action sequences. Its behavior depends on which states and transitions the dataset covers. A low training loss alone does not establish reliable rollout behavior.
- 2 At inference, observation changes alter the distribution used to sample the next action chunk. Receding-horizon execution creates opportunities to react after a disturbance, although the response is delayed until the next observation and prediction cycle.
- 3 The figure examines visual occlusion and physical perturbations during Push-T. These tests illustrate robustness in a specific setup. They do not guarantee recovery from disturbances or states absent from the evaluation.
- 4 OpenVLA predicts discrete action tokens. π_0 generates continuous chunks with a learned velocity field. Diffusion Policy generates chunks by conditional denoising. Representation, conditioning and sampling procedure explain the main algorithmic differences.



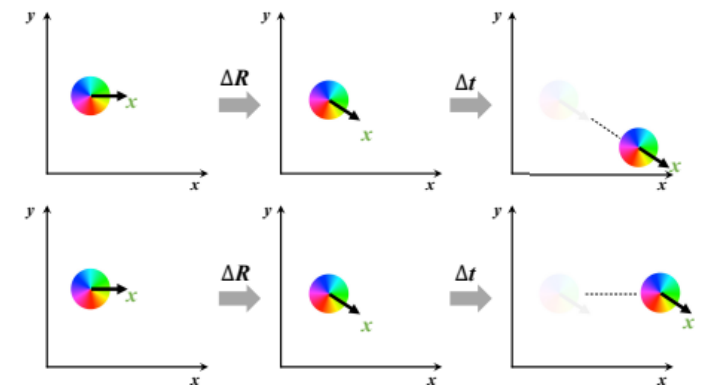
Chi et al., Diffusion Policy, Fig. 9. Occlusion and perturbation tests illustrate the interaction between feedback and learned action distributions.

Output representations, assumptions and failure modes

- 1 SAM 2 predicts a prompted pixel region and propagates object identity through memory. Ambiguous prompts, occlusion and accumulated errors can degrade masks. Evaluate region overlap and boundary quality.
- 2 FoundationPose estimates a rigid transform through pose hypotheses, rendering, refinement and ranking. It assumes consistent object geometry and RGB-D calibration. Symmetric objects may have equivalent poses.
- 3 ORB-SLAM3 estimates camera poses and landmarks from geometric constraints. Repeated texture, weak parallax and dynamic objects can corrupt correspondences. Loop verification checks whether distant matches are trustworthy.
- 4 OpenVLA predicts actions with an autoregressive token objective. Quantization limits resolution, while sequential decoding adds inference cost. Action normalization maps tokens back to continuous values.
- 5 π_0 learns a continuous velocity field. Diffusion Policy learns conditional denoising. Both sample action sequences conditioned on observations. Sampling steps, chunk length and execution horizon affect latency and feedback.
- 6 Masks, poses, trajectories and actions need different metrics. Compare methods on the same task and protocol. Ablations, uncertainty estimates and failure analysis explain more than one aggregate score.



Textureless / occluded object: observed image, ground truth and estimated pose. FoundationPose, Fig. 11.



Rotation and translation updates use different geometric conventions. FoundationPose, Fig. 10.